

ULLR · SECURITY REVIEW

ULLR Review.

A security review of Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
ULLR	Review	Guardian	June 30 – July 12, 2025	July 21, 2025

§ 00

Contents.

41 pages — overview, confidence ranking, scope, 28 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 21 findings	P. 09
	Remediation Review · 7 findings	P. 31
§ 05	Disclaimer	P. 40
§ 06	About Guardian	P. 41

§ 01

Overview.

Guardian reviewed Review across 2 review rounds. The final report includes 4 High, 4 Medium, 11 Low, 9 Informational findings.

AUDIT FIRM	Guardian	REPOSITORY	mitche50/ullr-protocol
CLIENT	ULLR	METHODOLOGY	Smart contract · Manual · Main Review · Remediation Review
PROJECT	Review	MAIN REVIEW COMMIT	566bcd45fc76
PREPARED BY	Guardian	REMEDATION REVIEW COMMIT	2e1a193990df
AUDIT WINDOW	June 30 – July 12, 2025		
FINAL REPORT	July 21, 2025		

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	4	0	0	0	4
Medium	4	0	0	0	4
Low	11	0	2	0	9
Informational	9	0	3	1	5
All severities	28	0	5	1	22

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Review in `mitche50/ullr-protocol`. The scope includes 596 source lines across 12 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/UllrLocker.sol	294	625
src/UllrFactory.sol	126	299
src/Errors.sol	33	102
src/DataTypes.sol	10	41
src/Constants.sol	15	56
src/implementations/UllrAerodromeImplementation.sol	67	136
src/interfaces/IUllrLocker.sol	20	38
src/interfaces/IUllrImplementation.sol	7	15
src/interfaces/IUllrFactory.sol	15	29
src/interfaces/IPoolFactory.sol	3	16
src/interfaces/IERC20.sol	3	9
src/interfaces/IAerodromePool.sol	3	5

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Locker can be drained by owner before lock end	Validation	HIGH	RESOLVED	p. 9
M-01	Re-orgs Allow For Locked Fund Theft	Reorgs	MEDIUM	RESOLVED	p. 11
M-02	getClaimableFees Inaccuracy For Initial LPs	Logical Error	MEDIUM	RESOLVED	p. 12
M-03	getClaimableFees Misses Accrued Fees	Logical Error	MEDIUM	RESOLVED	p. 13
L-01	User Locks May Be Overwritten	Validation	LOW	RESOLVED	p. 14
L-02	Upgrades Allowed For Claimable Locks	Unexpected Behavior	LOW	RESOLVED	p. 15
L-03	topUpLock Available For Disabled Implementations	Validation	LOW	RESOLVED	p. 16
L-04	Lacking Protocol Fee Slippage Check	Validation	LOW	RESOLVED	p. 17
L-05	Blacklist Token Claims Could Be Prevented	Warning	LOW	RESOLVED	p. 18
L-06	Emergency Withdraw Prevents Fee Collection	Warning	LOW	RESOLVED	p. 19
L-07	No minimum lock time or withdrawal delay	Best Practices	LOW	ACKNOWLEDGED	p. 20
L-08	Lack of upgrade compatibility checks	Validation	LOW	RESOLVED	p. 21
L-09	Ownership can be renounced	Access Control	LOW	RESOLVED	p. 22

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-01	Approval Risk	Warning	INFORMATIONAL	PARTIALLY RESOLVED	p. 23
I-02	Fees Claimed For Multiple Locks	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 24
I-03	topUpLock Can Still Deceive Users	Warning	INFORMATIONAL	RESOLVED	p. 25
I-04	Max Transfer Tokens Can Drain The Locker	Unexpected Behavior	INFORMATIONAL	RESOLVED	p. 26
I-05	Users Prevented From Upgrading	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 27
I-06	Lacking _getLockInfo Usage	Best Practices	INFORMATIONAL	RESOLVED	p. 28
I-07	computeLockerAddress can be made external	Gas Optimization	INFORMATIONAL	RESOLVED	p. 29
I-08	Emergency withdraws applied widely	Best Practices	INFORMATIONAL	RESOLVED	p. 30

REMEDIATION REVIEW

H-01	Permit transfers can permanently lock tokens	Frontrunning	HIGH	RESOLVED	p. 31
H-02	Owner can still exit lock early	Reentrancy	HIGH	RESOLVED	p. 32
H-03	Mismatching Interface	Logical Error	HIGH	RESOLVED	p. 34
M-01	Misformatted TypeHashes	Typo	MEDIUM	RESOLVED	p. 36
L-01	Missing contract code existence check	Best Practices	LOW	RESOLVED	p. 37
L-02	Missing nonReentrant Modifiers	Best Practices	LOW	ACKNOWLEDGED	p. 38
I-01	collectFeesForToken Uses Arbitrary Implementation	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 39

HIGH **H-01**

RESOLVED

Locker can be drained by owner before lock end

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/ullr-protocol-ullr-team1/blob/79ecae863c7b3ea891581d7b5acda60016c99353/src/implementations/UllrAerodromeImplementation.sol#L44>

REVIEW Main Review

DESCRIPTION

A locker owner can drain the contract of its locked tokens through reentrancy because the legitimacy of the Aerodrome LP is not checked correctly.

When a token is deposited via `lockTokens`, the implementation contract is called to check that the token is a legitimate Aerodrome LP token created by the Aerodrome factory. The code to check the token is as follows:

```
function supportsToken(address tokenContract) external view override returns (bool) {  
    return IAerodromePool(tokenContract).factory() == AERODROME_FACTORY;  
}
```

This does not effectively check that the token is an LP created from the Aerodrome factory as the token contract can simply create a `factory` function that returns the `AERODROME_FACTORY` address.

The locker contract can be drained in the following steps:

1. Owner creates a legitimate lock with legitimate token, X. The lock will last 24 months.
2. Owner creates an illegitimate lock with malicious token, Y. The lock period doesn't matter. `Y.token0()` will equal `token X` (the LP token). `Y.token1()` does not matter.
3. Owner calls `collectFees()` on lock for token Y. This calls token Y's `claimFees()` function which will call `UllrLocker.lockTokens()` with the legitimate token X and a short 1 second lock for the same amount as the first lock. The increased Locker balance of X will be distributed to the owner as fees, even though it's actually the amount of the new lock.
4. Owner waits one second and withdraws from the newest lock. The owner now has all of their token X.

Note that the owner would lose some value to protocol fees if the fee value is set to a positive value.

After further review, this attack could still take place even with legitimate Aerodrome LP pool tokens. An attacker could create an Aerodrome LP token using another Aerodrome LP

H-01 Locker can be drained by owner before lock end

CONTINUED

token as token0 and a token with transfer hooks as token1. Therefore, the LP token legitimacy check will not mitigate this vector.

— RECOMMENDATION

In the `supportsToken` function, make a call to the Aerodrome Factory's `isPool` function to verify the legitimacy of the LP token. Furthermore, add `nonReentrant` modifiers to state-changing functions such as `lockTokens` and `collectFees`.

Since Aerodrome has multiple pool factories, a call to the `FactoryRegistry.sol` contract must be made to ensure that the returned Factory value is legitimate via `isPoolFactoryApproved()`. Upon verification, then the specific pool must be verified via the `isPool` on the factory contract.

PROOF OF CONCEPT <https://gist.github.com/fatherGoose1/7d7c1920e36d727a2c88ab83f247b899>

— RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/9d7735f617ee48f09dfb0766672c4c2530b77454>

MEDIUM **M-01**

RESOLVED

Re-orgs Allow For Locked Fund Theft

CATEGORY Reorgs
LOCATION UllrFactory.sol: 107
REVIEW Main Review

DESCRIPTION

The arguments which determine the address of the locker contract are independent of the owner of the resulting locker. As a result, in the event of a chain re-org it may be possible for an attacker to steal funds which the user would have locked in their own locker.

Consider the following order of events:

- User A Creates a locker at address A in Block 100
- User A Creates a transaction to approve the locker and sends it to the mempool
- User B observes that a re-org is occurring and is able to create the same locker at address A (using the same salt), but with User B as the owner
- The re-org orders User B's transaction before Block 100 and User B owns the locker at address A
- User A's approval transaction still goes through and User B is able to steal the approved amount with the lockTokens function, passing User A as the from address

Furthermore, the creation of a locker could be DoS'd given that any actor can simply provide the same salt to create the locker address before a user's transaction is recorded.

RECOMMENDATION

Consider hashing the provided salt with the address of the owner so that each locker is unique to the owner address and therefore cannot be created at the same address with a different owner.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/62d3e22b84b9d48a6630217b08fe96d73e1ec96b>

MEDIUM **M-02**

RESOLVED

getClaimableFees Inaccuracy For Initial LPs

CATEGORY	Logical Error
LOCATION	UllrAerodromeImplementation.sol
REVIEW	Main Review

DESCRIPTION

In the `getClaimableFees` function when `userIndex == 0 && globalIndex > 0` is true then the users index is assigned to the global index, thereby ignoring any fees which have accumulated between the `userIndex` and the global index.

This is done to account for any LP token holders who have not yet been stamped with the latest index, however this should not be possible since the latest index is written in `_updateFor` on every token transfer in the `_beforeTokenTransfer` function.

As a result of this logic in the `getClaimableFees` function however, LPs which are the first provider when the index is zero will receive an inaccurate reporting of any fees generated before they update their position. For an initial LP locked the `getClaimableFees` view function will report zero and will not update until the fees are claimed or a new lock with the same token is made to update the index of the user to be nonzero.

RECOMMENDATION

Consider removing the handling for the `userIndex0 == 0 && globalIndex0 > 0` case since if an account holds an LP token balance it is updated to the latest index upon transfer.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/e555323730daac4e50ccce6576b091c7a6f38a82>

MEDIUM **M-03**

RESOLVED

getClaimableFees Misses Accrued Fees

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/ullr-protocol-ullr-team1/blob/79ecae863c7b3ea891581d7b5acda60016c99353/src/implementations/UllrAerodromeImplementation.sol#L131-L132
REVIEW	Main Review

DESCRIPTION

The `UllrAerodromeImplementation.sol` contract includes a function `getClaimableFees` that returns the fees that can be claimed from an Aerodrome pool contract. It recreates the reward logic by taking the difference of the locker contract's supply indexes compared to the pool's global supply indexes.

The issue is that it does not account for the locker contract's `claimable0` and `claimable1` mapping value. In the case where the supply checkpoints are updated for the locker contract through operations such as creating and new lock or topping up an existing lock, the `getClaimableFees` function will mistakenly return lower than expected values.

RECOMMENDATION

Include the locker contract's `claimable0` and `claimable1` values in the `getClaimableFees` function.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/7b81cd723923459f1dfbe2545f690ce3e70b42bc>

LOW **L-01**

RESOLVED

User Locks May Be Overwritten

CATEGORY	Validation
LOCATION	UllrLocker.sol
REVIEW	Main Review

— DESCRIPTION

In the rare case where 16,777,215 locks have been created, the nonce will overflow the 3 bytes allocated for the salt portion of the lock id. In this case it may be possible for a malicious actor to overwrite a user's lock if they're able to match the token, lockUntil, and withdrawalDelay of the lock and cause a lock id collision.

On chains like Ethereum this would be clearly cost prohibitive, however in some environments where gas is extremely cheap a malicious actor may be able to force such a collision to occur by creating many dummy locks with dust value.

— RECOMMENDATION

Validate if the resulting lockId in the lockTokens function already belongs to a lock entry with a nonzero amount entry, and if so, revert.

— RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/220ca3db7ec0ce6ade713e7f97882e6319ce1e29>

LOW **L-02**

RESOLVED

Upgrades Allowed For Claimable Locks

CATEGORY Unexpected Behavior
LOCATION UllrLocker.sol
REVIEW Main Review

DESCRIPTION

In the `upgradeImplementation` function the `withdrawalAllowedAt` is validated to be zero to ensure that locks which have already had their withdrawals triggered cannot be upgraded.

However for locks which are technically claimable and have a withdrawal delay of zero, they can still be upgraded while being immediately claimable. This is because the `_requireWithdrawable` function skips the `withdrawalAllowedAt` validation if the lock withdrawal delay is 0.

RECOMMENDATION

Consider using the `_isWithdrawable` function to validate whether a lock is immediately claimable in the `upgradeImplementation` function.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/a2f4aa31cce1693ffddafad874d8a21e34e6b4c6>

LOW **L-03**

RESOLVED

topUpLock Available For Disabled Implementations

CATEGORY	Validation
LOCATION	UllrLocker.sol
REVIEW	Main Review

DESCRIPTION

In the `UllrLocker` contract the `topUpLock` function does not validate if the implementation contract of the lock is still enabled in the factory before allowing the user to add more to their lock.

RECOMMENDATION

Consider if this is the expected behavior, or if the `topUpLock` function should use the `isImplementationRegistered` function to validate that the implementation is indeed active.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/42867c2ae9132b48610e734204c2d24c2b792eca>

LOW **L-04**

RESOLVED

Lacking Protocol Fee Slippage Check

CATEGORY Validation
LOCATION UllrFactory.sol
REVIEW Main Review

DESCRIPTION

The default protocol fee is assigned as 25 basis points and the `MAX_PROTOCOL_FEE` is assigned as 1000 basis points, therefore there is a wide range of fees which may be charged depending on when the `UllrLocker` is created.

However there is no slippage validation to ensure that the `_protocolFee` value used to create the locker clone with immutable args is the value that the creator expected it to be when they originally transmitted the transaction.

As a result, the owner of the `UllrFactory` contract may frontrun transactions that invoke the `createLocker` function and unexpectedly raise the protocol fee amount to the maximum value with the `setProtocolFee` function. Unsuspecting users may not realize this and end up paying the maximum 1000 basis points fee for the entire duration of their lock when they instead intended to accept a much lower fee rate.

RECOMMENDATION

Consider adding a fee slippage parameter which allows the user to specify what threshold of fee they are willing to accept.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/c1aefc2fe3017d8867ad15ef483bda8f2cea6e1c>

LOW **L-05**

RESOLVED

Blacklist Token Claims Could Be Prevented

CATEGORY Warning
LOCATION Global
REVIEW Main Review

DESCRIPTION

The owner of the `UllrFactory` contract may assign a fee receiver which happens to be a blacklisted address for one of the fee tokens and therefore DoS's all withdrawals and fee claims.

RECOMMENDATION

Be aware of this, if this edge case is desired to be mitigated to remove all necessary trust then consider allowing the `protocolFeeAmount` to be made claimable to the `feeReceiver` if the token transfer fails instead of reverting with `safeTransfer`.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/e91e1bc829cdb7f35cdaecbd908f60934bed4f0b>

LOW **L-06**

RESOLVED

Emergency Withdraw Prevents Fee Collection

CATEGORY Warning
LOCATION UllrLocker.sol
REVIEW Main Review

DESCRIPTION

The emergency withdrawal flow removes the LP tokens from the locker contract without claiming the fees for the LP, which is intended. However now the fees for the lock that was withdrawn cannot be claimed since the `lockInfo` struct has been deleted for the lock.

There are ways of rescuing the fees by creating another lock for the contract for the same LP token, however it may be preferable to add a separate `collectFees` function which allows the owner to claim fees outside of a specific lock.

RECOMMENDATION

Consider adding an `onlyOwner` `collectFees` function which allows the owner to collect fees for a given token, irrespective of the lock.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/652e58a67aba34f917f56df1ac7d2db4a73e1174>

LOW **L-07**

ACKNOWLEDGED

No minimum lock time or withdrawal delay

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/ullr-protocol-ullr-team1/blob/79ecae863c7b3ea891581d7b5acda60016c99353/src/UllrLocker.sol#L90>

REVIEW Main Review

— DESCRIPTION

The only requirement for a lock's total duration (lock duration + withdrawal delay) is that at least one of the values must be non-zero. This allows total lock durations as little as 1 second.

— RECOMMENDATION

Consider adding a minimum duration check, for example, 1 month.

— RESOLUTION

Status: Acknowledged.

LOW **L-08**

RESOLVED

Lack of upgrade compatibility checks

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/ullr-protocol-ullr-team1/blob/79ecae863c7b3ea891581d7b5acda60016c99353/src/UllrLocker.sol#L219-L232
REVIEW	Main Review

DESCRIPTION

The locker owner can upgrade the implementation contract used for a token deposit. However, when upgrading the implementation, it does not perform the same sanity checks as upon initial locking. This can result in reverts due to incompatibility when attempting to claim fees as part of the withdrawal process.

RECOMMENDATION

Implement the `supportsToken` check in `upgradeImplementation`.

```
if (!IUllrImplementation(implementation).supportsToken(token)) {  
    revert UllrLocker__UnsupportedToken();  
}
```

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/d9faec4c3e0b3d93ce7798b5c0924ba4bf63d286>

LOW **L-09**

RESOLVED

Ownership can be renounced

CATEGORY Access Control

LOCATION <https://github.com/GuardianOrg/ullr-protocol-ullr-team1/blob/79ecae863c7b3ea891581d7b5acda60016c99353/src/UllrLocker.sol#L22>

REVIEW Main Review

DESCRIPTION

The `UllrLocker.sol` contract inherits from `Ownable.sol`, meaning it contains functions for transferring and renouncing ownership. In the case that ownership is accidentally or maliciously renounced, the LP tokens will remain locked in the contract forever.

RECOMMENDATION

Consider overriding `renounceOwnership()` and revert.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/30a1c37af646efa74af6b5eae09ea9afa994bba1>

INFORMATIONAL **I-01**

PARTIALLY RESOLVED

Approval Risk

CATEGORY	Warning
LOCATION	UllrLocker.sol
REVIEW	Main Review

DESCRIPTION

In the `UllrLocker` contract `safeTransferFrom` is used with an arbitrary from address indicating that users must approve the `UllrLocker` contract prior to performing their lock or topUp action through an integrating protocol.

It is crucial that this approval is made in the same transaction in which the lock or topUp action is carried out by the integrating protocol, otherwise depending on the integration, the approval amount could be used to fund another lock made by a malicious user.

RECOMMENDATION

Be sure to document this risk for integrating protocols who would choose to expose the Ullr locking mechanism to their users.

RESOLUTION

Status: Partially resolved.

<https://github.com/mitche50/ullr-protocol/commit/84a24a2a820f12f33c3c73cbcd4362695696d2>

INFORMATIONAL **I-02**

ACKNOWLEDGED

Fees Claimed For Multiple Locks

CATEGORY	Warning
LOCATION	UllrLocker.sol
REVIEW	Main Review

— DESCRIPTION

In the withdrawal flow the `_collectAndProcessFees` function is used to claim fees to the owner address, however since the fee claim is not made based upon how much the amount of the lock is fees from other locks using the same underlying pool will also be claimed and sent to the owner.

This may simply be unexpected for the owner using the locker contract and also needs to be accounted for in any integrating protocol using Ullr as a locking partner to avoid fees being stolen by other users or a DoS of the `minFeeAmounts` validation.

— RECOMMENDATION

Let this finding document this potentially unexpected behavior for users and integrating protocols.

— RESOLUTION

Status: Acknowledged.

<https://github.com/mitche50/ullr-protocol/commit/1712160367d298a467f49c7f91115a50def6bde>
e

INFORMATIONAL I-03

RESOLVED

topUpLock Can Still Deceive Users

CATEGORY Warning
LOCATION UllrLocker.sol
REVIEW Main Review

DESCRIPTION

In the NatSpec for the `topUpLock` function it is mentioned that the validation ensures that the owner `Cannot top up locks where withdrawal is possible to prevent misleading users.`

However the `topUpLock` function can still be called when the user's lock duration has passed, but they have not yet triggered their withdrawal. The withdrawal delay assigned for the lock can be a very short period, a number of seconds, and allow the user to `topUpLock` and `triggerWithdrawal` to withdraw very shortly afterwards.

RECOMMENDATION

Users can still verify the state of the lock and withdrawal delay on-chain, consider if the current behavior is acceptable or if the validation should be changed to ensure that the users lock is not expired.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/7cd3a2c90d9a101d2edbafa49beeeb1e93d1bf09>

INFORMATIONAL **I-04**

RESOLVED

Max Transfer Tokens Can Drain The Locker

CATEGORY	Unexpected Behavior
LOCATION	UllrLocker.sol
REVIEW	Main Review

DESCRIPTION

Some ERC20 tokens such as cUSDCv3 have the behavior that transferring the maximum `type(uint256).max` value transfers the entire account balance instead.

This may create unexpected issues depending on how protocols use the Ullrlocker, where the user specifies `type(uint256).max` as the amount and the from address only transfers a small amount of tokens. A malicious actor can create a targeted overflow using this method in the `topUpLock` function which allows them to assign the amount of their lock to the entire token balance of the contract and steal tokens which were not theirs.

RECOMMENDATION

Consider using the token amount which was received in the `topUpLock` and `lockTokens` functions instead of the amount specified by the user.

Otherwise be aware and document the fact that the locker system should be incompatible with such tokens.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/c996ee8d74a5da90140478b6f9c11ce17a195c92>

INFORMATIONAL I-05

ACKNOWLEDGED

Users Prevented From Upgrading

CATEGORY Warning
LOCATION UllrLocker.sol
REVIEW Main Review

DESCRIPTION

The `upgradeImplementation` function validates that the `withdrawalAllowedAt` is not greater than zero to prevent users who have triggered withdrawals from upgrading.

This bears some risk in that users who have a long withdrawal delay cannot upgrade their underlying protocol implementation contract in case a security issue is discovered in it.

RECOMMENDATION

Be aware of this risk and consider if it is desired.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-06

RESOLVED

Lacking `_getLockInfo` Usage

CATEGORY	Best Practices
LOCATION	UllrLocker.sol
REVIEW	Main Review

DESCRIPTION

Throughout the `UllrLocker` contract the `_locks` mapping is directly accessed, however the `_getLockInfo` function already exists to access the `_locks` mapping in a safe way with the appropriate empty struct check.

RECOMMENDATION

Consider constricting access of the `_locks` mapping directly to just the `_getLockInfo` function and using the `_getLockInfo` to access the lock where necessary in the `UllrLocker` contract.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/2b4d710f4d7115035c79745954638389a87f2e01>

INFORMATIONAL **I-07**

RESOLVED

computeLockerAddress can be made external

CATEGORY	Gas Optimization
LOCATION	https://github.com/GuardianOrg/ullr-protocol-ullr-team1/blob/79ecae863c7b3ea891581d7b5acda60016c99353/src/UllrFactory.sol#L120-L123
REVIEW	Main Review

DESCRIPTION

The `computeLockerAddress` function is not called inside the `UllrFactory.sol` contract, and can thus be made `external` to save some gas when calling it.

RECOMMENDATION

Set `computeLockerAddress` to external visibility.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/5fd38fe3ebbd6ca2b39e771765ded66ad6f0b1d4>

INFORMATIONAL I-08

RESOLVED

Emergency withdraws applied widely

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/ullr-protocol-ullr-team1/blob/79ecae863c7b3ea891581d7b5acda60016c99353/src/UllrLocker.sol#L259>

REVIEW Main Review

DESCRIPTION

The Ullr admin has the ability to enable emergency withdrawals for specific logic contracts. This is inflexible in the case where a specific LP or underlying token has an issue, the locker owner will not be able to withdraw it.

RECOMMENDATION

Adding an emergency withdraw function specific to the locker would mitigate issues where withdrawals cannot be completed due to issues specific to LP tokens or the underlying token transfers.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/71501cc4a16cfee18064af8eb7106ba03449976f>

HIGH **H-01**

RESOLVED

Permit transfers can permanently lock tokens

CATEGORY	Frontrunning
LOCATION	https://github.com/GuardianOrg/ullr-protocol-ullr-team1/blob/cb7658b46dc754b7230a80799b0fdb6bd304803/src/UllrLocker.sol#L271
REVIEW	Remediation Review

DESCRIPTION

The `lockTokensWithPermit` and `topUpLockWithPermit` functions were added to save users from having to perform an approval transactions prior to locking tokens. However, these two functions do not verify the intent of the permit which can result in permanently frozen funds for any permits granted to this contract.

A malicious user can monitor the mempool and frontrun transactions to either of these functions. The attacker can supply a nearly infinite lock time, permanently freezing the funds in the contract.

RECOMMENDATION

Require another signature provided to both functions that verifies the intent of the action. Construct a permit type hash for each action and also hash the individual parameters of the new lock or top up, such as lock duration and withdrawal delay.

Verify that the signer of the hash is the `from` address.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/3306b8694115d0410071873f1db9591dd7dcd464>

HIGH **H-02**

RESOLVED

Owner can still exit lock early

CATEGORY	Reentrancy
LOCATION	https://github.com/GuardianOrg/ullr-protocol-ullr-team1/blob/cb7658b46dc754b7230a80799b0fdb6bd304803/src/implementations/UllrAerodromeImplementation.sol#L100
REVIEW	Remediation Review

— DESCRIPTION

H-01 was partially fixed by properly verifying that the LP token used is a legitimate Aerodrome LP token. However, there is still a vector for the owner to be able to withdraw their LP early.

The owner can use the Aerodrome pool factory to create a legitimate LP using another Aerodrome LP as token0 a custom owner-controlled ERC20 with hooks as token1.

In the same way outlined in H-01, when token1 is transferred through the fee claiming mechanism, control will be transferred to the owner's token contract which can then lock more LP (token0) with a 1 second lock.

Since the LP (token0) is transferred to the Locker contract during the fee claim, the Locker contract transfers all of the new LP (token0) to the owner. Then the owner can withdraw their new lock 1 second later, bypassing all locks in the process.

Steps:

- Owner locks LPX in Locker.sol
- Owner deploys MaliciousToken with transfer hook
- Owner creates LPY via the Aerodrome PoolFactory which takes LPX as token0 and MaliciousToken as token1
- Owner locks some LPY in Locker.sol
- Owner calls `collectAndProcessFees` on LPY. When MaliciousToken is transferred to the Locker contract, it will also call `Locker.lockTokens()` to lock LPX for only 1 second.
- The extra LPX deposited will be accounted for as claimed fees and sent to the owner.
- Owner withdraws the newest lock 1 second later.

— RECOMMENDATION

Add reentrancy guards to all user-facing functions.

H-02 Owner can still exit lock early

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/2e1a193990df2f574040b7b93e09118bcae48ef3>

HIGH **H-03**

RESOLVED

Mismatching Interface

CATEGORY	Logical Error
LOCATION	—
REVIEW	Remediation Review

DESCRIPTION

The interface used in the Ullr codebase for the `permitTransferFromWithAdditionalDataERC20` function is as follows:

```
function permitTransferFromWithAdditionalDataERC20(  
    address token,  
    uint256 nonce,  
    uint256 permitAmount,  
    uint256 expiration,  
    address owner,  
    address to,  
    uint256 transferAmount,  
    bytes memory signedPermit,  
    bytes memory additionalData,  
    bytes32 typeHash  
)  
external  
returns (bool isError);
```

However the interface that is available in the [deployed PermitC contract] (<https://basescan.org/address/0x721c008fdff27bf06e7e123956e2fe03b63342e3#code>) is:

```
function permitTransferFromWithAdditionalDataERC20(  
    address token,  
    uint256 nonce,  
    uint256 permitAmount,  
    uint256 expiration,  
    address owner,  
    address to,  
    uint256 transferAmount,  
    bytes32 additionalData,  
    bytes32 advancedPermitHash,
```

H-03 Mismatching InterfaceCONTINUED

```
bytes calldata signedPermit  
) external returns (bool isError);
```

As a result these interfaces do not match up with the PermitC contract available onchain.

— RECOMMENDATION

Consider updating the interface used to match the PermitC contracts onchain.

— RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/cc4ba2082692fd0c57e141bab7bedab17d5fedfa>

MEDIUM **M-01**

RESOLVED

Misformatted TypeHashes

CATEGORY	Typo
LOCATION	Constants.sol
REVIEW	Remediation Review

DESCRIPTION

The `LOCK_TOKENS_TYPEHASH` and `TOP_UP_LOCK_TYPEHASH` are missing the function name and opening parenthesis at the beginning of the string.

RECOMMENDATION

Include the relevant beginning for:

```
LOCK_TOKENS_TYPEHASH: keccak256("address token,uint256 amount,uint256 withdrawalDelay,uint256 lockPeriod,address implementation")
```

```
TOP_UP_LOCK_TYPEHASH: keccak("bytes32 lockId,uint256 amount")
```

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/cc4ba2082692fd0c57e141bab7bedab17d5fedfa>

LOW **L-01**

RESOLVED

Missing contract code existence check

CATEGORY Best Practices
LOCATION UllrLocker.sol: 785
REVIEW Remediation Review

DESCRIPTION

In the `_processFees` function if the token transfer provided no return data the success of the transfer is assumed to be true. However in the event that the target token contract does not house bytecode this will errantly indicate that a transfer has successfully occurred for a token that does not exist.

It is unlikely that this case would arise with the current underlying implementation, however future implementation contracts may allow this edge case to arise.

RECOMMENDATION

Validate that the token address houses bytecode in the `_processFees` function.

RESOLUTION

Status: Resolved.

<https://github.com/mitche50/ullr-protocol/commit/34c664cd01cb6a9cab9fa6c9a5d143ff60459ae2>

LOW **L-02**

ACKNOWLEDGED

Missing nonReentrant Modifiers

CATEGORY	Best Practices
LOCATION	UllrLocker.sol
REVIEW	Remediation Review

— DESCRIPTION

The following external functions lack nonReentrant modifiers:

- triggerWithdrawal
- cancelWithdrawalTrigger

Although no explicit exploit path has been identified with these functions, out of an abundance of caution you may consider adding nonReentrant modifiers to these functions as well.

— RECOMMENDATION

Consider adding `nonReentrant` modifiers to these functions.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

collectFeesForToken Uses Arbitrary Implementation

CATEGORY	Warning
LOCATION	UllrLocker.sol
REVIEW	Remediation Review

— DESCRIPTION

The `collectFeesForToken` function allows the caller to supply their own implementation contract which must be an active implementation which supports the token passed.

This allows for potentially unexpected edge cases where a locker owner uses a different implementation to claim their fees than the one that is actively associated with one or more ongoing locks for that token.

This same edge case also applies when a locker owner holds multiple locks for the same token with different implementation versions being used.

— RECOMMENDATION

Simply be aware of this edge case and consider it for future iterations of the implementations available for lockers.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.